

Script-Controlled ACL – Restrict Record Access Based on Field Value

Naan Mudhalvan / Skill Wallet Project - Development Phase Document

1. Project & Team Information

- Project Title: Script-Controlled ACL – Restrict Record Access Based on Field Value
- Phase: Development Phase (Implementation & Scripting)
- Team ID: SWTID-2026-9611
- Team Leader: Adhsa Rani P
- Team Members: Gayathri A, Parkavi P, Priyadharshini M, Chinthanika P
- Platform: ServiceNow / SkillWallet (SmartBridge)

2. Development Scope & Environment Setup

- Target Table: `u_institution_details`
- Development Environment: ServiceNow Developer Instance
- Custom Roles Configured: `bb1`, `bb2`, `bb3`, `bb4`
- Application Package: Scoped Application / Update Set (`Script-Controlled ACL Implementation`)

3. Step-by-Step Configuration & Implementation

3.1 Role & Table Configuration

1. Created custom roles: `bb1` (Viewer), `bb2` (Contributor), `bb3` (Editor), `bb4` (Administrator).
2. Created custom table `u_institution_details` with key fields:
 - `u_branch` (String / Choice: EEE, CSE, ECE, MECH)
 - `state` (Choice: Draft, Active, Closed)
 - `is_confidential` (True/False)

3.2 Access Control List (ACL) Rules Implementation

A. Delete ACL Rule Configuration

- Type: `record`
- Operation: `Delete`
- Name / Table: `u_institution_details`

- Active: `true`
- Requires Role: `bb4`
- Script Code:

```
(function executeRule(current, previous /*null when async*/) {

    // 1. Explicit Role Verification

    if (!gs.hasRole('bb4')) {

        return false;

    }

    // 2. Field-Based Dynamic Restriction

    if (current.u_branch == 'EEE') {

        return true; // Grant delete permission for EEE branch records

    }

    return false; // Fallback deny for non-EEE branch records

})();
```

B. Read ACL Rule Configuration

- Type: `record`
- Operation: `Read`
- Name / Table: `u_institution_details`
- Active: `true`
- Requires Role: `bb1`
- Script Code:

```
(function executeRule(current, previous) {

    if (current.is_confidential == true && !gs.hasRole('bb3')) {
```

```

        return false; // Hide confidential records from lower tier roles
    }

    return true;

})();

```

C. Write ACL Rule Configuration

- Type: `record`
- Operation: `Write`
- Name / Table: `u_institution_details`
- Active: `true`
- Requires Role: `bb3`
- Script Code:

```

(function executeRule(current, previous) {

    if (current.state == 'Closed') {

        return false; // Prevent record edits when state is Closed

    }

    return true;

})();

```

4. Code Structure & Best Practices Applied

- **Optimized Performance:** Avoided `GlideRecord` database queries inside ACL scripts to ensure ultra-fast list view rendering.
- **Session Cache Utilization:** Utilized `gs.hasRole()` and `gs.getUser()` cached methods alongside inline `current` object fields.
- **Fail-Safe Fallback:** Structured all logical paths to default to `return false;` in compliance with Zero-Trust principles.

5. Development Verification Summary

- **Delete Operation:** Successfully restricted to users with `bb4` role acting on records where `u_branch == 'EEE'`.
- **Write Operation:** Successfully locked editing on records in `Closed` state.

- **Read Operation:** Seamlessly filtered confidential records in list views and form views.